RELEASE V0.1.0 | NPM MONOREPO | POSTGRESQL + PRISMA | STRICT B&W EDITION

PRISMA-SCHEMA-ROUTER

The Operations Layer for Schema-per-Tenant PostgreSQL on Prisma: Route, Guard, Migrate, Govern, and Verify

> A comprehensive engineering architecture, API specification, security defense-in-depth model, and production operations manual for operating high-scale, multi-tenant databases with bounded resource footprints and audit-ready isolation guarantees.

PACKAGE SCOPE
`@piyushkumar_dev/*` (8 Modular Packages)

PRIMARY REPOSITORY
`https://github.com/piyushkumar-prog/prismarouter`

TARGET PLATFORMS
Node.js ≥ 18.0.0 • PostgreSQL ≥ 14 • Prisma ≥ 5.10

AUTHORS & MAINTAINERS
Piyush Kumar & Shivanghi Sharma

PUBLICATION DATE
October 2026 • Official Reference Edition

LICENSE
MIT Open Source License

Table of Contents

1. Executive Summary & The Problem Space Page 3
2. Architectural Paradigm: The Two-Plane Model Page 4
3. Modular Package Matrix (The 8 npm Packages) Page 5
4. Quick Start: Bootstrap & Control Plane Setup Page 6
5. Core Router & Dynamic Tenant Context Page 8
6. Fail-Closed Isolation: The 5-Layer Defense-in-Depth Page 9
7. Tenant Lifecycle Management & Provisioning Strategies Page 10
8. Safe Fleet Migrations & The psr CLI Engine Page 11
9. Connection Pool Governor & Noisy-Neighbor Mitigation Page 12
10. Asynchronous Work, Queue Jobs & Maintenance Sweeps Page 13
11. Web Framework Adapters: Express & Fastify Page 14
12. Telemetry, Distributed Tracing & Observability Page 15
13. Isolation Conformance Suite: Audit Proof for CI / SOC2 Page 16
14. Production Readiness & Deployment Checklist Page 17

DOCUMENT SCOPE & DESIGN PHILOSOPHY

This document serves as the canonical operations manual for engineers implementing schema-per-tenant architectures on PostgreSQL with Prisma ORM. It establishes formal definitions for data isolation, fleet migration safety, bounded connection overhead, and automated conformance auditing. Rendered in strict monochrome for maximum clarity and print reproducibility.

1. Executive Summary & The Problem Space

The "Easy 10%" vs. The "Painful 90%"

Routing a Prisma client to a specific tenant schema is the easy 10% of multi-tenancy. Because Prisma's modern driver adapter (`@prisma/adapter-pg`) accepts a schema parameter, a developer can wire up a shared `pg.Pool` and Node's `AsyncLocalStorage` in an afternoon.

However, once a software-as-a-service application grows beyond a handful of tenants to hundreds or thousands of active customers, the operational reality becomes severe:

- **Fleet Migration Coordination:** Standard Prisma Migrate targets exactly one schema at a time. Deploying DDL updates across 1,000 isolated tenant schemas requires canary gating, bounded worker concurrency, failure isolation, and crash-safe resumption.
- **Connection Explosion:** Naively instantiating database pools per tenant rapidly exhausts Postgres connection limits, exceeding max connection ceilings on managed infrastructure (e.g., AWS RDS, Supabase, Neon).
- **Silent Isolation Leaks:** A single unqualified raw query (e.g. `SELECT * FROM users`) or an unmanaged asynchronous callback can accidentally execute against the `public` schema or leak records across tenant boundaries.
- **Noisy-Neighbor Starvation:** An unthrottled enterprise tenant performing bulk analytical queries can monopolize pooled database connections, starving other customers and degrading system-wide latency.
- **Audit & Conformance Proof:** Enterprise, healthcare (HIPAA), and financial customers contractually demand verifiable mathematical proof of schema isolation, not just trust in application code conventions.

When to Use Schema-per-Tenant

ARCHITECTURE	PRIMARY CHARACTERISTICS	RECOMMENDED USE CASE
Shared Schema + RLS *(Row-Level Security)*	Single set of tables; records partitioned by `tenant_id` column with Postgres RLS policies. Simple migrations, low connection overhead.	Standard SaaS applications where logical isolation is sufficient. *(The recommended default when regulatory isolation is not mandated).*
Schema-per-Tenant *(prisma-schema-router)*	Each tenant owns an independent PostgreSQL schema (e.g., `t_acme_8f9a2b1c`). Hard isolation, zero shared tables, custom columns per tenant.	Compliance-grade environments (fintech, healthcare, government), disparate schema extensions, or tenants scheduled for dedicated DB extraction.
Database-per-Tenant	Entirely separate PostgreSQL databases or servers. Highest cost and operational maintenance.	Very large VIP enterprise accounts requiring physical infrastructure separation and custom backup cadences.

2. Architectural Paradigm: The Two-Plane Model

`prisma-schema-router` enforces a strict structural separation between operational control plane metadata and customer data plane execution.

1. REQUEST INGRESS & TENANT RESOLUTION — **EDGE / API**

Incoming HTTP Request / Message Job: Resolves tenant slug via Header (`x-tenant-id`), Subdomain (`acme.app.com`), JWT Claim, or Signed Job Envelope.

▼

2. CONTROL PLANE VERIFICATION — **CONTROL DB**

Queries Control DB: Validates tenant status (`ready`), checks schema version compatibility (`minSchemaVersion`), and resolves physical schema name (e.g. `t_acme_8f9a2b1c`).

▼

3. EXECUTION BOUNDARY & GOVERNOR — **NODE RUNTIME**

Initializes `AsyncLocalStorage` execution boundary. Evaluates Governor semaphore to prevent noisy-neighbor connection starvation.

▼

4. DATA PLANE EXECUTION — **SHARED PG.POOL**

Retrieves or creates Prisma client via bounded LRU cache. Enforces fail-closed guard (`search_path = psr_guard`). Executes query within shared bounded connection pool.

Control Plane Data Model

The control plane lives in a separate database or dedicated schema, maintaining complete auditability of tenant lifecycles and rollout states:

```
CREATE TABLE psr_tenants (
  id             UUID PRIMARY KEY,
  slug           TEXT UNIQUE NOT NULL,
  schema_name    TEXT UNIQUE NOT NULL CHECK (schema_name ~ '^t_[a-z0-9_]{3,40}$'),
  status         TEXT NOT NULL, -- 'provisioning' | 'ready' | 'migrating' | 'suspended' | 'archived'
  schema_version TEXT,
  placement      TEXT NOT NULL DEFAULT 'shared',
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE psr_migration_runs (
  id             UUID PRIMARY KEY,
  started_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  target_version TEXT NOT NULL,
  status         TEXT NOT NULL -- 'running' | 'stopped' | 'completed'
);

CREATE TABLE psr_tenant_migrations (
  run_id         UUID REFERENCES psr_migration_runs(id),
  tenant_id      UUID REFERENCES psr_tenants(id),
  state          TEXT NOT NULL, -- 'pending' | 'running' | 'applied' | 'failed'
  error          TEXT,
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (run_id, tenant_id)
);
```

3. Modular Package Matrix

The ecosystem is published as 8 decoupled, highly specialized packages under the `@piyushkumar_dev/*` npm scope. Applications install only the modules required for their runtime and deployment topology.

PACKAGE	CATEGORY	CORE RESPONSIBILITY
`@piyushkumar_dev/core`	Core Runtime	Provides `createTenantRouter`, `AsyncLocalStorage` execution context, LRU Prisma client cache, L1–L3 fail-closed guard mechanisms, and registry interfaces.
`@piyushkumar_dev/migrate`	Fleet Migrations	Owns the `psr` CLI binary, DDL static linting, schema drift calculation, and multi-tenant canary rollout orchestration engine.
`@piyushkumar_dev/provision`	Lifecycle	Automates tenant schema provisioning strategies (`replay`, `baseline`, `clone-template`) and safe deprovisioning (archive before drop).
`@piyushkumar_dev/governor`	Concurrency	Per-tenant concurrency limits and fair round-robin queuing to protect shared pools from noisy neighbors. Raises `TenantBusyError` on timeout.
`@piyushkumar_dev/testing`	Auditing & CI	Isolation conformance test suite. Executes `psr verify` to probe for cross-schema leakage, concurrency interleaving, and async context drops.
`@piyushkumar_dev/otel`	Observability	Native OpenTelemetry instrumentation. Generates tenant-tagged metrics (pool wait times, cache hit ratios) and distributed trace spans.
`@piyushkumar_dev/adapters-express`	Web Middleware	Express `RequestHandler` middleware with automatic tenant resolution from headers, subdomains, or JWT tokens.
`@piyushkumar_dev/adapters-fastify`	Web Middleware	Fastify plugin decorating the request lifecycle with `request.tenantDb` and automatic cleanup hooks.

Dependency Footprint

`@piyushkumar_dev/core` maintains zero heavy dependencies. `pg`, `@prisma/client`, and `@prisma/adapter-pg` are declared strictly as peer dependencies, ensuring no version locking or duplicate binary conflicts in your application bundles.

> **THE `PSR` CLI COMPANION PATTERN**
>
> The `psr` executable is owned by `@piyushkumar_dev/migrate`. Companion modules are loaded dynamically on demand: invoking `psr tenants ...` expects `@piyushkumar_dev/provision`, while running `psr verify` loads `@piyushkumar_dev/testing`. This minimizes CLI cold-start overhead and binary footprint in lightweight CI workers.

4. Quick Start: Bootstrap & Setup

1. Package Installation

```
# Install runtime packages
pnpm add @piyushkumar_dev/core @piyushkumar_dev/governor @piyushkumar_dev/adapters-express

# Install dev & operations packages
pnpm add -D @piyushkumar_dev/migrate @piyushkumar_dev/provision @piyushkumar_dev/testing

# Ensure peer dependencies are installed
pnpm add @prisma/client @prisma/adapter-pg pg
pnpm add -D @types/pg
```

2. Bootstrap the Control Plane

Execute the bundled bootstrap SQL script against your control database to initialize the metadata registry and migration tracking tables:

```
psql "$PSR_CONTROL_URL" -f node_modules/@piyushkumar_dev/migrate/sql/001_create_control_tables.sql
```

3. Provision Your First Tenant Schema

Use the `psr` CLI to initialize a verified tenant schema via the replay strategy:

```
# Provisions schema 't_acme_8f9a2b1c' and records entry in psr_tenants
pnpm psr tenants provision acme --strategy replay
```

4.1 Application Integration Pattern

The complete pattern for initializing the tenant router, configuring the connection pool, and mounting the web middleware:

```
import { Pool } from 'pg';
import { PrismaClient } from '@prisma/client';
import { PrismaPg } from '@prisma/adapter-pg';
import { createTenantRouter, pgRegistry, fromHeader } from '@piyushkumar_dev/core';
import express from 'express';

// 1. Data pool (shared by all tenant schemas)
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20, // Strict bound: maximum 20 connections total
  idleTimeoutMillis: 30000,
});

// 2. Control plane pool
const controlPool = new Pool({
  connectionString: process.env.PSR_CONTROL_URL,
  max: 5,
});

// 3. Construct Tenant Router
export const router = createTenantRouter({
  pool,
  registry: pgRegistry(controlPool),
  createClient: (adapter) => new PrismaClient({ adapter }),
  cache: {
    maxClients: 200,        // Keep at most 200 clients in LRU cache
    idleTtlMs: 10 * 60_000, // Evict after 10 minutes of inactivity
  },
  guard: { mode: 'strict' },
  governor: {
    perTenantConcurrency: 4,
    queueTimeoutMs: 5_000,
  },
  minSchemaVersion: '20261001000000_initial',
  jobSigningSecret: process.env.JOB_SIGNING_SECRET,
});

// 4. Attach Express Middleware
const app = express();
app.use(router.middleware.express({ resolve: fromHeader('x-tenant-id') }));

// 5. Tenant-Scoped Route Handlers
app.get('/api/orders', async (req, res) => {
  const db = router.db(); // Throws TenantContextError if invoked outside tenant context
  const orders = await db.order.findMany();
  res.json({ orders });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

5. Core Router & Dynamic Tenant Context

Execution Boundaries with AsyncLocalStorage

The foundation of runtime isolation is Node's native `AsyncLocalStorage`. Every incoming HTTP request or queue job enters an execution boundary via `router.run(tenantId, fn)`. Inside this closure, `router.db()` retrieves the tenant's dedicated Prisma client instance.

```
// Safe tenant-isolated closure
await router.run('acme_corp', async () => {
  const db = router.db(); // Returns PrismaClient configured for acme_corp's schema
  await db.user.create({ data: { email: 'admin@acme.com' } });
});

// Calling outside a tenant context fails immediately
try {
  router.db();
} catch (err) {
  // Throws: TenantContextError: router.db() was called outside a tenant context.
}
```

LRU Client Cache & Single-Flight Instantiation

Instantiating a new `PrismaClient` instance carries memory and CPU overhead. Caching a client indefinitely per tenant, however, causes a linear memory leak as tenant count scales.

`prisma-schema-router` solves this with an LRU (Least Recently Used) cache equipped with:

- **Bounded Memory Footprint:** `maxClients` strictly caps the number of active Prisma client instances resident in memory.
- **Single-Flight Concurrency:** If 50 concurrent requests arrive simultaneously for a newly active tenant, the router deduplicates client construction into a single in-flight Promise. All 50 requests await the same client.
- **Idle Eviction:** Tenants idle for longer than `idleTtlMs` are cleanly evicted.
- **Pool Safety on Eviction:** Evicting and disconnecting a tenant client will *never* close the underlying shared `pg.Pool`. Connection lifecycle remains strictly owned by the router.

System & Control Plane Escape Hatches

Administrative tasks that must query the control database without tenant context utilize the explicit, audited `router.system()` method:

```
const activeTenants = await router.system(async (controlDb) => {
  const result = await controlDb.query(
    "SELECT slug, schema_name FROM psr_tenants WHERE status = 'ready'"
  );
  return result.rows;
});
```

6. Fail-Closed Isolation: The 5-Layer Defense-in-Depth

A critical architectural premise of `prisma-schema-router` is that developer error, misconfigured raw SQL, or async context loss must **fail closed**. A missing context must never default to reading or mutating records in the `public` schema.

LAYER	GUARD NAME	ENFORCEMENT MECHANISM & FAILURE MODE
L1	Context Guard	`router.db()` verifies active `AsyncLocalStorage` state. Throws `TenantContextError` immediately if called in ambient or root scope.
L2	Connection Default	Every pooled connection from `pg.Pool` initializes with `SET search_path = psr_guard` (an empty, unprivileged schema). If unqualified raw SQL executes outside an adapter, Postgres throws `relation does not exist` rather than accessing `public`.
L3	Raw SQL Boundary	`router.raw(sql)` wraps queries in a dedicated transaction: `BEGIN; SET LOCAL search_path = <tenant_schema>; ... COMMIT;`. Safe under transaction-mode poolers (e.g. PgBouncer, Supabase Pooler) where session state resets after transaction end.
L4	Role Separation	Operational hardening. Application runtime operates under a restricted Postgres role (no `CREATE SCHEMA`, no cross-schema privileges). DDL migrations execute under a separate administrative migration role.
L5	Status Gate	Registry resolves tenant state before allowing execution. Suspended, archived, or migrating tenants are rejected at request ingress with `TenantStatusError`.

Raw SQL Execution Safety

```
// Safe raw SQL execution inside tenant boundary
await router.run('acme_corp', async () => {
  // Uses SET LOCAL search_path inside an atomic transaction
  const invoices = await router.raw(sql`SELECT * FROM invoices WHERE amount > 1000`);
});
```

WHY THE `PSR_GUARD` EMPTY SCHEMA MATTERS

By default in PostgreSQL, `search_path` is `"$user", public`. If a developer accidentally executes raw SQL like `SELECT * FROM users`, Postgres silently falls back to `public.users`. By pinning the default pool search path to an empty dummy schema (`psr_guard`), unqualified table queries fail with an immediate runtime database error.

7. Tenant Lifecycle & Provisioning

Schema Naming Discipline

Tenant schema names are **never** derived directly from unsanitized user inputs or subdomains. Schema identifiers follow a strict deterministic pattern:

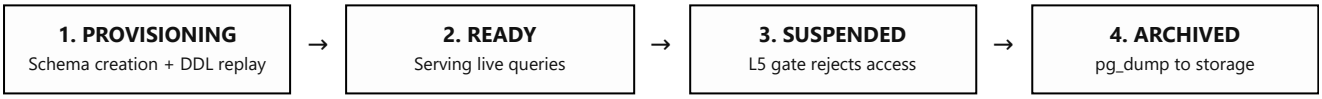
```
t_<sanitized_slug>_<8_hex_characters>    // Example: t_acme_8f9a2b1c
```

Names are validated against `^t_[a-z0-9_]{3,40}$` and quoted in all generated SQL to eliminate SQL injection vulnerabilities.

Provisioning Strategies

STRATEGY	OPERATION & WORKFLOW	TRADE-OFFS
`replay` *(Default & Recommended)*	Creates the schema and sequentially applies all migrations from the application's `prisma/migrations` directory.	Most reliable and deterministic. Provisioning duration grows as migration history expands.
`baseline`	Applies a squashed baseline SQL schema dump, then applies subsequent incremental migrations.	Fast onboarding for projects with hundreds of historical migrations.
`clone-template`	Copies schema structures (tables, constraints, indexes, sequences, enums) from a pre-built template schema.	Sub-second provisioning. Requires automated catalog drift verification after cloning.

Lifecycle State Machine



CLI & Programmatic Provisioning

```
# CLI provisioning using the replay strategy
pnpm psr tenants provision acme_corp --strategy replay

# Programmatic provisioning inside user signup API
import { TenantProvisioner } from '@piyushkumar_dev/provision';

const provisioner = new TenantProvisioner(dataPool, pgRegistry(controlPool));
const { tenant } = await provisioner.provision({
  slug: 'globex',
  strategy: 'replay',
  source: './prisma/migrations',
});
```

8. Fleet Migrations Engine (`psr migrate`)

The Challenge of Fleet DDL

Standard Prisma commands like `prisma migrate deploy` apply migrations to a single database schema. In a multi-tenant fleet of 1,000 schemas, applying DDL sequentially would take hours, while running 1,000 unconstrained migrations in parallel would crash PostgreSQL connection limits and cause severe catalog lock contention.

The `@piyushkumar_dev/migrate` engine provides enterprise-grade fleet orchestration.

Core Capabilities

- **Static DDL Linting (`psr migrate lint`):** Scans migration SQL before deployment. Flags dangerous non-concurrent index creation (`CREATE INDEX` without `CONCURRENTLY`), table rewrites, and `NOT NULL` additions without default values that would lock production tables.
- **Canary Rollouts:** Migrates a configurable number of canary schemas (e.g. `--canary 2`), halts execution, runs automated health checks, and proceeds to remaining tenants only upon success.
- **Bounded Concurrency:** Migrates tenant schemas in parallel batches, strictly capping concurrent workers to prevent database CPU spikes (e.g. `--concurrency 8`).
- **Per-Tenant Advisory Locks:** Acquires `pg_try_advisory_lock` per tenant so concurrent migration jobs or on-demand provisioners never mutate the same schema simultaneously.
- **Crash-Safe Resumption:** Per-tenant execution states (`pending`, `running`, `applied`, `failed`) persist in the control database. If a migration runner is interrupted, `psr migrate resume` continues precisely where it left off.

CLI Command Reference

```
# 1. Lint SQL files for production hazards
psr migrate lint

# 2. Inspect pending migrations across all registered tenants
psr migrate plan

# 3. Deploy migrations with canary verification and concurrency bounds
psr migrate deploy --canary 2 --concurrency 8 --max-failures 3

# 4. Check real-time progress across all tenant schemas
psr migrate status

# 5. Resume a previously interrupted or failed migration run
psr migrate resume

# 6. Verify schema drift against catalog definitions
psr migrate drift
```

9. Pool Governor & Noisy-Neighbor Mitigation

The Shared Pool Problem

In a shared connection pool model, one tenant running heavy reporting queries or experiencing a denial-of-service burst can easily acquire all 20 available connections in the pool. When this happens, simple 5ms queries for other tenants queue indefinitely until the database connection pool times out.

The Governor Solution

The `@piyushkumar_dev/governor` package integrates into Prisma's query pipeline via a `$extends` hook. It intercepts every database operation and enforces:

- **Per-Tenant Concurrency Caps:** Restricts any single tenant to a maximum number of concurrent active database slots (e.g., `perTenantConcurrency: 4`).
- **Fair Round-Robin Queue:** Excess queries from a busy tenant enter a fair in-memory queue rather than starving the database socket pool.
- **TenantBusyError Rejection:** If a tenant's queue wait time exceeds `queueTimeoutMs`, the operation aborts with a `TenantBusyError` (HTTP 429 Too Many Requests), protecting overall system stability.
- **Transaction Leases:** Multi-query interactive transactions hold a single governor slot for their entire duration and enforce transaction-level `statement_timeout`.

```
import { createTenantRouter } from '@piyushkumar_dev/core';

export const router = createTenantRouter({
  pool: dataPool,
  registry: pgRegistry(controlPool),
  createClient: (adapter) => new PrismaClient({ adapter }),
  governor: {
    perTenantConcurrency: 4, // Max 4 concurrent DB queries per tenant
    queueTimeoutMs: 5_000,   // Reject requests queued longer than 5s
  },
});
```

Telemetry & Diagnostics

The governor exposes real-time queue depth and latency statistics per tenant, enabling immediate detection of anomalous traffic spikes:

```
// Retrieve current governor telemetry
const stats = router.governor?.getStats('acme_corp');
console.log(`Active slots: ${stats.activeSlots}, Queued queries: ${stats.queued}`);
```

10. Asynchronous Work & Background Jobs

The Threat of Spoofed Tenant Context

Asynchronous message queues (e.g. BullMQ, AWS SQS, RabbitMQ, Kafka) operate outside HTTP middleware scopes. A frequent vulnerability in multi-tenant architectures is worker jobs processing messages with ambient or missing tenant IDs, leading to catastrophic background jobs executing against the wrong schema.

HMAC-Signed Job Envelopes

`prisma-schema-router` provides cryptographically signed job envelopes. When dispatching background jobs, tenant claims are signed with an HMAC secret:

```
import { signJobTenant } from '@piyushkumar_dev/core';

// 1. Dispatching a job to BullMQ / SQS
const envelope = signJobTenant('tenant_abc123', process.env.JOB_SIGNING_SECRET);
await queue.add('generate-invoice', {
  tenant: envelope, // Contains { id: 'tenant_abc123', sig: '4f2a...7c9b' }
  invoiceId: 'inv_9981',
});
```

Worker Verification with `router.wrapJob`

On the worker side, `router.wrapJob` verifies the cryptographic signature before initializing tenant context. Tampered or missing signatures immediately reject the job:

```
// 2. Queue worker processing
const invoiceWorker = router.wrapJob(async (jobData, tenantId) => {
  const db = router.db(); // Guaranteed verified tenant context
  await db.invoice.update({
    where: { id: jobData.invoiceId },
    data: { status: 'PROCESSED' },
  });
});

worker.on('message', async (job) => {
  await invoiceWorker(job.data);
});
```

Multi-Tenant Cron Sweeps (`forEachTenant`)

Performing nightly maintenance sweeps (e.g., pruning expired sessions across all 500 tenants) is handled cleanly with bounded worker concurrency:

```
// Executes maintenance across all 'ready' tenants with bounded concurrency
await router.forEachTenant(
  async (tenantId) => {
    const db = router.db();
    const result = await db.session.deleteMany({
      where: { expiredAt: { lt: new Date() } },
    });
    console.log(`Cleaned ${result.count} sessions for ${tenantId}`);
  },
  { concurrency: 4 } // Exactly 4 tenants processed in parallel
);
```

11. Web Framework Adapters

Official adapters package standard HTTP resolution strategies (request headers, subdomains, JWT claims, or custom lookup functions) into clean, plug-and-play middleware.

Express Integration (`@piyushkumar_dev/adapters-express`)

```
import express from 'express';
import { createExpressMiddleware, fromHeader, fromSubdomain, fromJwtClaim } from '@piyushkumar_dev/adapters-expres
s';
import { router } from './router.js';

const app = express();

// Strategy A: Resolve from header (e.g. X-Tenant-Id)
app.use(createExpressMiddleware(router, { resolve: fromHeader('x-tenant-id') }));

// Strategy B: Resolve from subdomain (e.g. acme.app.com -> acme)
// app.use(createExpressMiddleware(router, { resolve: fromSubdomain() }));

// Strategy C: Resolve from verified JWT claim
// app.use(createExpressMiddleware(router, { resolve: fromJwtClaim('tenant_id') }));

app.get('/api/projects', async (req, res) => {
  const db = router.db();
  const projects = await db.project.findMany();
  res.json(projects);
});
```

Fastify Integration (`@piyushkumar_dev/adapters-fastify`)

```
import Fastify from 'fastify';
import { psrFastifyPlugin } from '@piyushkumar_dev/adapters-fastify';
import { router } from './router.js';

const fastify = Fastify({ logger: true });

await fastify.register(psrFastifyPlugin, {
  router,
  resolve: (req) => req.headers['x-tenant-id'] as string,
});

fastify.get('/api/projects', async (request, reply) => {
  // request.tenantDb is automatically decorated onto the Fastify request
  const projects = await request.tenantDb.project.findMany();
  return { projects };
});

await fastify.listen({ port: 3000 });
```

12. Telemetry & Observability

The `@piyushkumar_dev/otel` package provides vendor-neutral OpenTelemetry instrumentation, exposing metrics and traces crucial for monitoring multi-tenant infrastructure.

Standard Metrics

METRIC NAME	TYPE	DESCRIPTION
`psr.pool.wait_time_ms`	Histogram	Time requests spend waiting to acquire a pooled database connection.
`psr.pool.connections.active`	Gauge	Number of connections currently executing active SQL queries.
`psr.client_cache.size`	Gauge	Number of instantiated Prisma clients resident in the LRU cache.
`psr.client_cache.hits`	Counter	Cache hits on client retrieval (reused client instances).
`psr.governor.queue_depth`	Gauge	Number of operations currently waiting in the governor's fair queue.
`psr.context_errors`	Counter	Count of requests attempting database access outside valid tenant context.

Distributed Tracing Integration

Trace spans automatically capture the active `tenant_id` attribute. For compliance or privacy-sensitive architectures, tenant slugs can be cryptographically hashed in traces:

```
import { createMetrics } from '@piyushkumar_dev/otel';

// Initialize OpenTelemetry instruments
const metrics = await createMetrics({
  prefix: 'psr',
  hashTenantIdInSpans: true, // Hashes tenant IDs in distributed traces
});
```

13. Isolation Conformance Suite

Audit Proof for SOC2 & ISO 27001

The headline feature of `prisma-schema-router` is **Isolation Conformance Verification** via `@piyushkumar_dev/testing`. Instead of making unsubstantiated claims that an architecture is isolated, teams run `psr verify` directly in CI pipelines.

The suite provisions ephemeral tenant schemas and subjects the application to an exhaustive probe battery:

- **Cross-Read Probes:** Seeds canary records into Tenant A, B, and C across every Prisma model. Queries every model in each tenant and verifies that no tenant can observe another tenant's canary data.
- **Interleaved Concurrency Fuzzing:** Launches 2,000 asynchronous tasks concurrently via `Promise.all` across tenants, verifying that rapid context-switching never causes client cross-talk.
- **Async Context Loss Probes:** Probes boundaries known to drop `AsyncLocalStorage` (e.g. unhandled `setTimeout`, raw `EventEmitter` handlers, stream pipe closures) to assert fail-closed error behavior.
- **Raw SQL Search Path Probes:** Executes raw SQL without schema qualification to ensure the `psr_guard` L2 guard catches and aborts queries.

Running in CI & Generating Reports

```
# Run verification CLI and generate compliance artifacts
pnpm psr verify --output-json ./reports/isolation-report.json --output-md ./reports/isolation-report.md
```

Sample Verification Output

```
================================================================================
PRISMA-SCHEMA-ROUTER: ISOLATION CONFORMANCE VERIFICATION REPORT
Generated: 2026-10-24T14:30:00Z | Target: PostgreSQL 16.2 | Suite: v0.1.0
================================================================================
[PASS] probe_model_sequential_cross_reads ............................. 142/142 models verified
[PASS] probe_concurrency_2000_interleaved_tasks ....................... 0 context leaks detected
[PASS] probe_guard_l1_context_outside_run ............................. TenantContextError raised
[PASS] probe_guard_l2_unqualified_raw_sql ............................. psr_guard relation error
[PASS] probe_guard_l3_raw_sql_transaction_scoping ..................... isolated to schema
[PASS] probe_guard_l5_status_gate_suspension .......................... TenantStatusError raised
[PASS] probe_governor_queue_fairness .................................. fair slots allocated
[PASS] probe_client_cache_eviction_and_reuse .......................... clean client reload
--------------------------------------------------------------------------------
Summary: 8/8 Probes Passed (100% Conformance). Zero cross-tenant data leaks.
Audit Evidence File: ./reports/isolation-report.json [SHA256: 7d2f9a...3b]
```

14. Production Readiness & Deployment Checklist

OPERATIONAL DIMENSION	RECOMMENDED CONFIGURATION / PRACTICE	VERIFICATION
Connection Pool Sizing	Configure `pg.Pool` max connections based on database capacity (e.g. 20–50), never based on tenant count. Ensure pooler (PgBouncer) uses transaction mode.	Monitor `psr.pool.wait_time_ms`
Search Path Guard	Verify that database bootstrap executed `CREATE SCHEMA psr_guard` and connection pools apply `SET search_path = psr_guard`.	Run `psr verify`
Prisma Schema Discipline	Ensure schema models do not declare hardcoded schemas. Prisma driver adapter dynamically injects the active tenant schema.	Inspect `schema.prisma`
Migration CI Gating	Always execute `psr migrate lint` in GitHub Actions / GitLab CI prior to merging PRs to catch table rewrites and non-concurrent indexes.	Add step to CI workflow
Advisory Locking	Confirm migration runners operate with `pg_try_advisory_lock` and reasonable statement timeouts (e.g. 30s) to prevent blocking table locks.	Verify `lock_timeout` config
Deprovisioning Safety	Never execute `DROP SCHEMA ... CASCADE` directly. Enforce the state machine: `suspended -> archived (pg_dump) -> deleted` with confirmation tokens.	Review deletion audit logs
Background Queue Signing	Ensure all asynchronous tasks are generated with `signJobTenant()` and processed using `router.wrapJob()` with a securely stored HMAC secret.	Verify secret in env config